C. COMPUTAÇÃO
UNIPAC

CENTRO UNIVERSITÁRIO UNIPAC
SCIENTIA LIBERTAS ET FIDES
PRESIDENTE ANTÔNIO CARLOS

unipac.br
Barbacena

Computação Gráfica

LUZ E SOMBRA

ILUMINAÇÃO

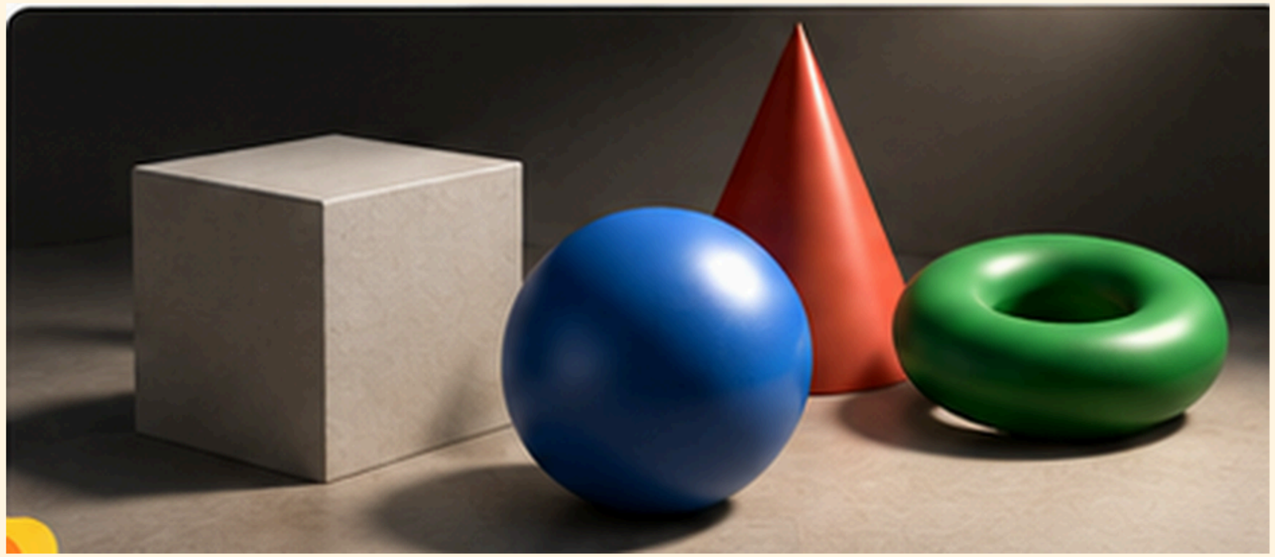
Professor:

Dr. Rodrigo Fenandes dos Santos

COMPUTAÇÃO GRÁFICA

Luz e sombra na Computação Gráfica

Como a iluminação modifica a aparência dos objetos 3D?



A iluminação permite:

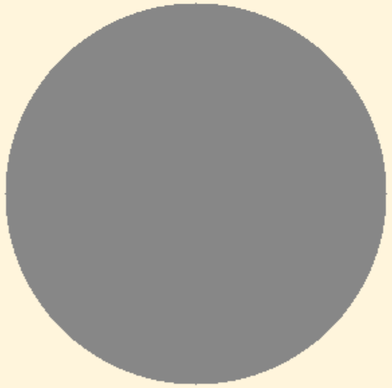
- Destacar formas e volumes
- Perceber materiais
- Criar profundidade
- Tornar a cena mais realista
- Guiar o olhar do observador

A luz ajuda a compreender a forma e a superfície dos objetos.

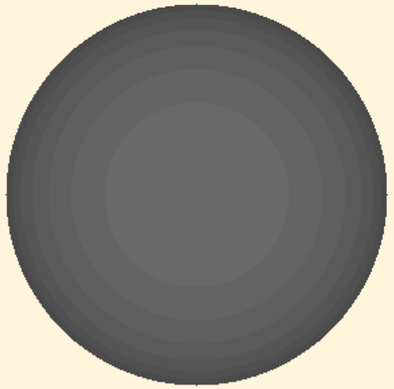
Por que a iluminação importa?

Uma mesma esfera pode apresentar diferentes resultados visuais.

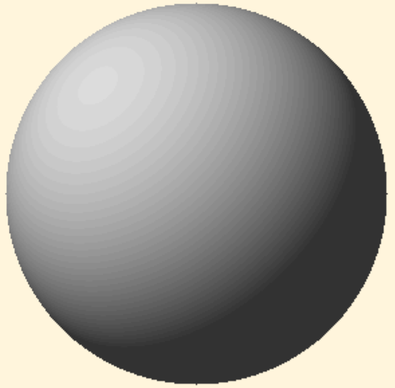
Sem luz aplicada



Luz ambiente



Ambiente + difusa



Ambiente + difusa + especular

A iluminação revela volume, orientação e características do material.

Iluminação e percepção das superfícies

O material determina como a luz aparece sobre a superfície.

Fosco

Metálico

Plástico

Reflexivo

Superfícies diferentes interagem com a luz de modos diferentes.

Iluminação ambiente

Uma contribuição uniforme que ajuda a visualizar toda a cena.

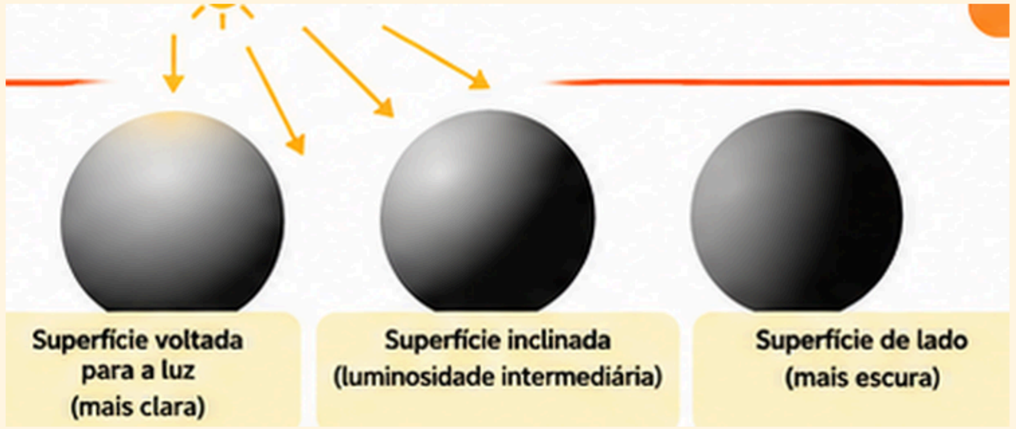
- Afeta igualmente os pontos das superfícies.
- Simula a iluminação indireta do ambiente.
- Não possui uma direção específica.
- Evita regiões completamente escuras.

A luz ambiente fornece uma luminosidade básica à cena.

Iluminação difusa

A intensidade muda conforme a orientação da superfície.

- A luz é espalhada em várias direções.
- A face voltada para a luz fica mais clara.
- Faces inclinadas recebem menos luz.
- O vetor normal indica a orientação.



A luz difusa ajuda a perceber a orientação das superfícies.

COMPUTAÇÃO GRÁFICA

Iluminação especular

Produz destaques de brilho em superfícies mais polidas.

- Representa o brilho intenso em materiais polidos.
- Depende da posição do observador.
- Aparece como um destaque localizado.
- Quanto mais polida a superfície, mais intenso e concentrado é o brilho.

Pouco especular

Mais especular

A iluminação especular ajuda a caracterizar materiais brilhantes.

COMPUTAÇÃO GRÁFICA

Comparando os três tipos de iluminação

A combinação ambiente, difusa e especular produz imagens mais realistas.

Apenas ambiente

Apenas difusa

Apenas especular

Ambiente + difusa + especular

Cada componente de iluminação contribui de forma diferente para a aparência final.

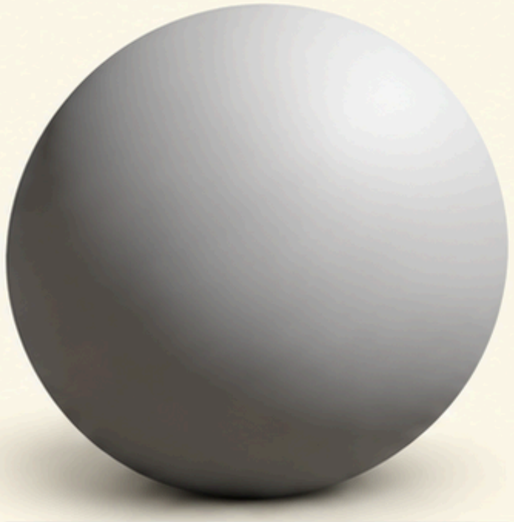
COMPUTAÇÃO GRÁFICA

Modelos de iluminação: Lambert e Phong

Diferentes modelos produzem aparências distintas para os materials.

Modelo de Lambert

Combina a iluminação ambiente e difusa.



Superfície fosca
(sem destaque especular)

- Considera as componentes ambiente e difusa.
- Produz superfícies foscas e com aparência suave.
- Não gera destaque especular.
- Simples e eficiente.

Modelo de Phong

Combina as componentes ambiente, difusa e especular.

Superfície brilhante
(com destaque especular)

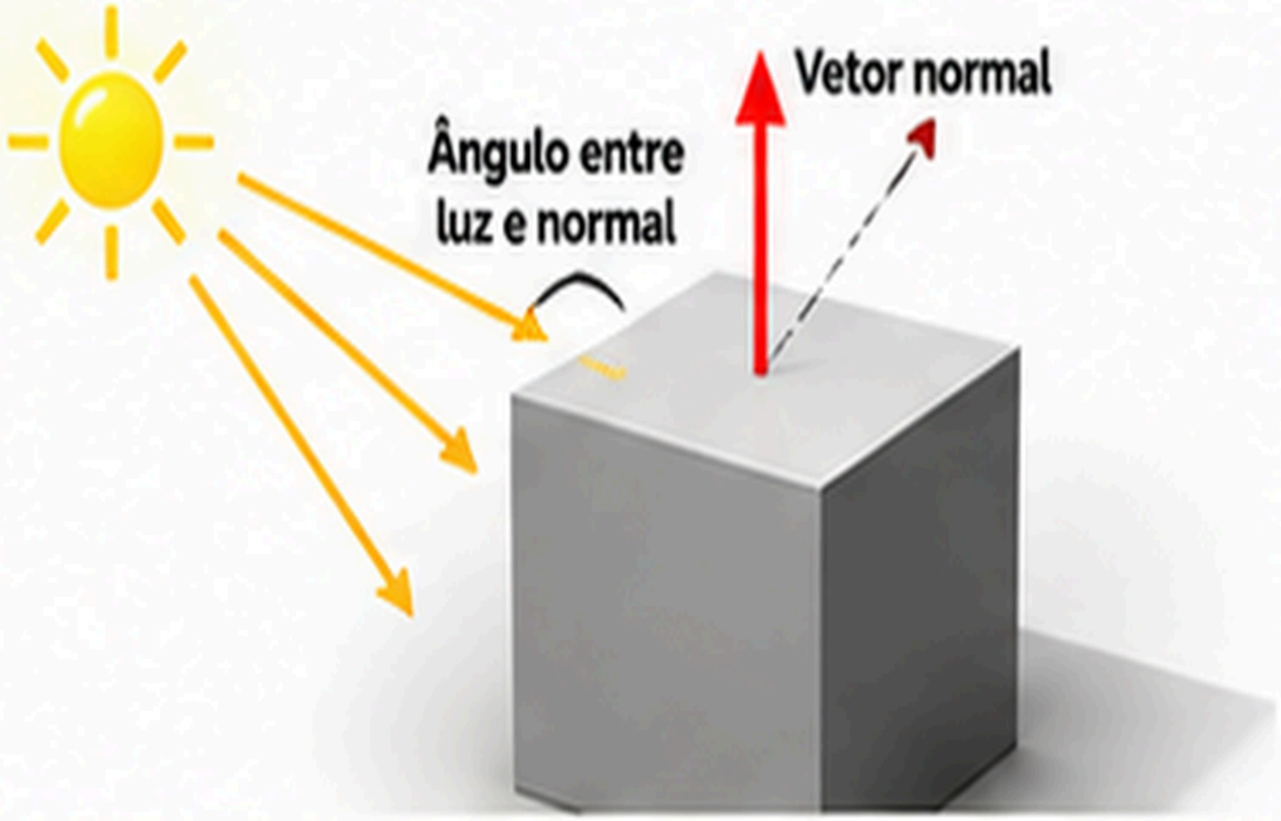
- Inclui as componentes ambiente, difusa e especular.
- Produz superfícies mais realistas e brilhantes.
- Gera destaque especular cuja posição depende do observador.
- Adequado para materiais polidos.

O modelo de Phong acrescenta a componente especular, produzindo destaques de brilho.

COMPUTAÇÃO GRÁFICA

Direção da luz e vetores normais

A intensidade da iluminação depende da orientação da superfície.



- A superfície recebe mais luz quando o ângulo é menor.
- A intensidade diminui conforme o ângulo aumenta.
- O vetor normal indica a orientação da superfície.
- Superfícies voltadas para longe da luz recebem menos iluminação.

A orientação da superfície em relação à luz influencia diretamente a sua aparência.

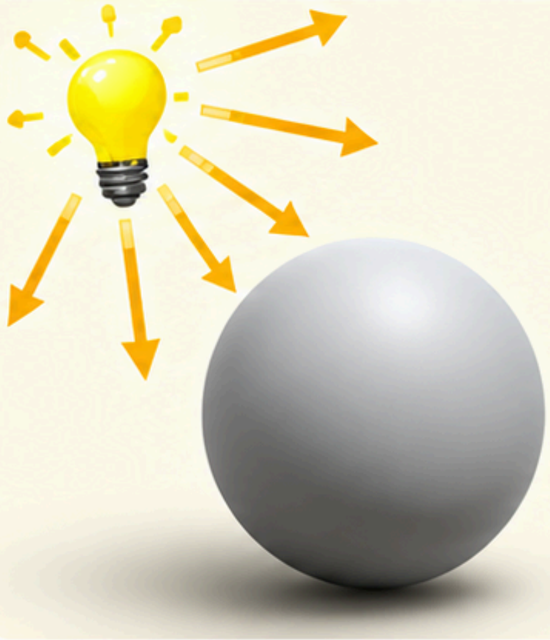
COMPUTAÇÃO GRÁFICA

Luz pontual e luz direcional

Duas fontes de luz com comportamentos diferentes.

Luz pontual

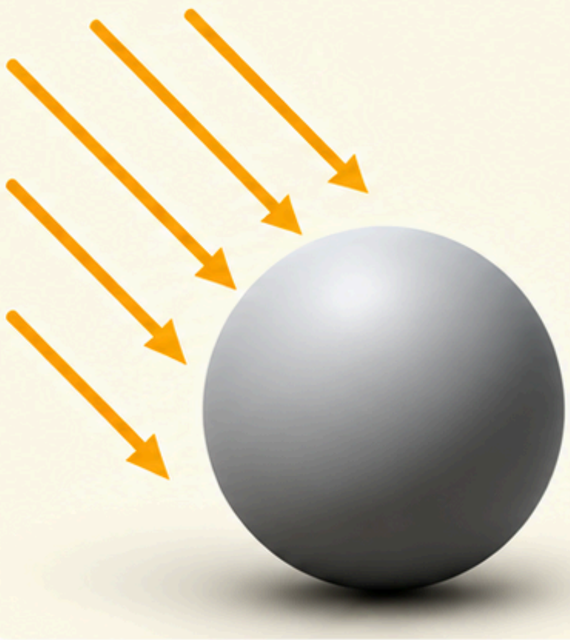
Emite luz a partir de um ponto específico no espaço.



- Emite luz a partir de um ponto no espaço.
- A intensidade diminui com a distância.
- Ilumina em todas as direções.
- Simula fontes como lâmpadas e lanternas.

Luz direcional

Simula uma fonte de luz muito distante.



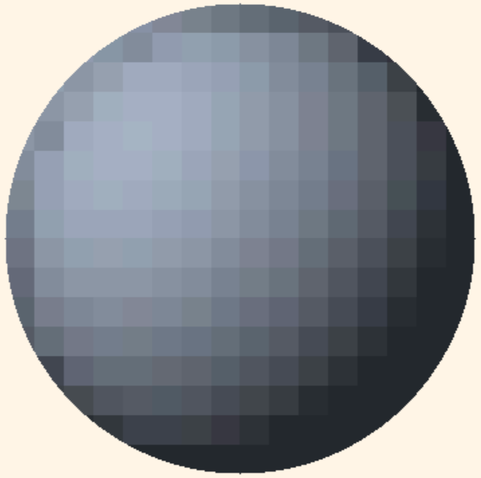
- Luz com direção constante.
- Raios paralelos.
- Não possui atenuação com a distância.
- Simula fontes distantes, como o Sol.

A luz pontual simula fontes próximas, enquanto a luz direcional representa fontes distantes.

Modelos de sombreamento

Como a iluminação é calculada altera a aparência dos objetos.

Flat

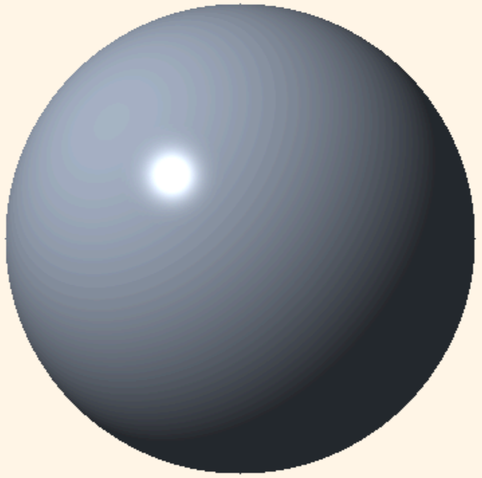


Normal por face. Aparência facetada.

Gouraud

Cálculo por vértice, com interpolação.

Phong



Normais interpoladas; iluminação por fragmento.

Flat evidencia faces; Gouraud e Phong suavizam o resultado.

COMPUTAÇÃO GRÁFICA

Da iluminação ao código

As classes de luz encapsulam os parâmetros de iluminação no framework.

- Cada tipo de luz possui **características específicas.**
- As classes permitem configurar **cor, tipo, direção, posição e atenuação.**
- Os objetos de luz podem ser **adicionados à cena** como qualquer outro objeto 3D.
- Essa estrutura facilita a **manutenção e a expansão** do framework.

Organização em classes

O framework utiliza classes para representar os diferentes tipos de luz, facilitando a criação, o controle e a reutilização nas cenas 3D.

Light
Classe base com atributos comuns.

AmbientLight
Representa a iluminação ambiente.

DirectionalLight
Representa uma fonte de luz direcional.

PointLight
Representa uma fonte de luz pontual.

COMPUTAÇÃO GRÁFICA

Classe base Light

Define os atributos comuns a todos os tipos de luz.

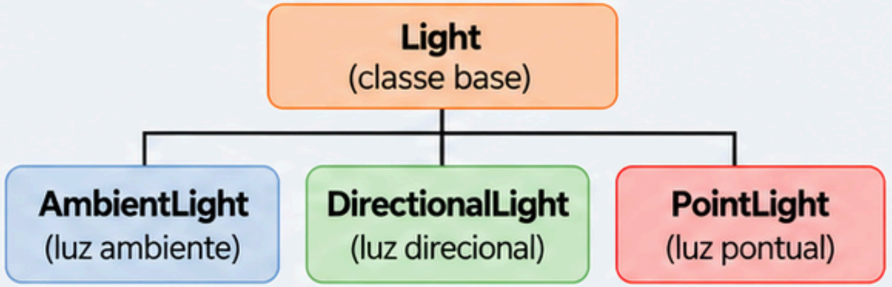
```python
class Light(Object3D):
    AMBIENT = 1
    DIRECTIONAL = 2
    POINT = 3

    def __init__(self, lightType=0):
        self.lightType = lightType
        self.color = [1, 1, 1]
        self.attenuation = [1, 0, 0]
```

`lightType`	identifica o tipo de luz (ambiente, direcional ou pontual).
`color`	armazena a cor RGB da luz.
`attenuation`	define os coeficientes de atenuação da luz.

Principais características

- É a classe base comum para todos os tipos de luz do framework.
- Centraliza os atributos compartilhados entre as luzes.
- Suporta luzes ambiente, direcional e pontual por meio do atributo `lightType`.
- Seus dados são utilizados durante o processo de renderização da cena.

Estrutura da hierarquia



A classe Light organiza os dados comuns, facilitando a criação de luzes específicas.

COMPUTAÇÃO GRÁFICA

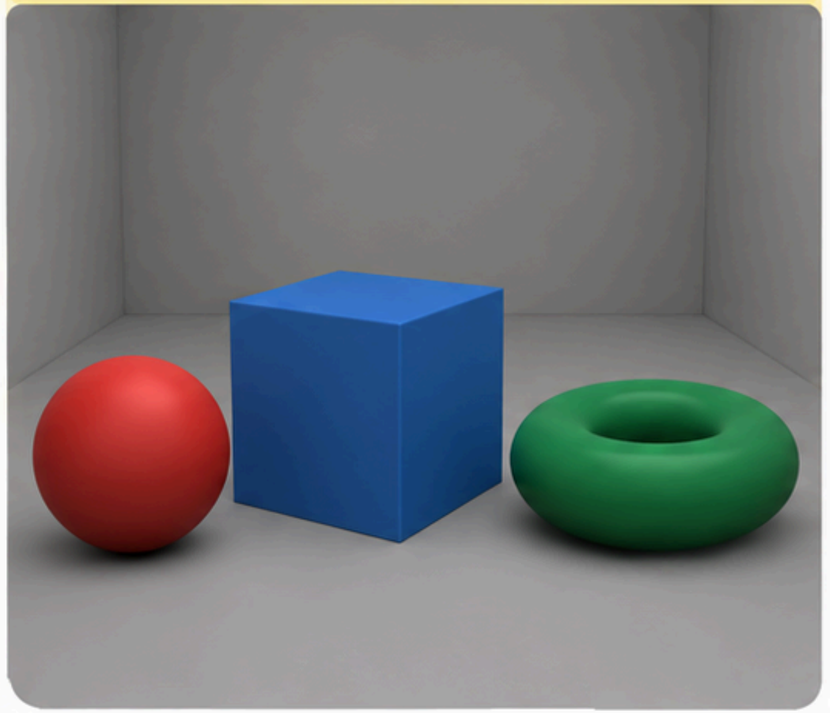
Luz ambiente em Python

Representa a contribuição uniforme de iluminação na cena.

Classe da luz ambiente

```python
class AmbientLight(Light):
    def __init__(self, color=[1, 1, 1]):
        super().__init__(Light.AMBIENT)
        self.color = color


# Exemplo de uso
luz_ambiente = AmbientLight([0.4, 0.4, 0.4])
```

Somente iluminação ambiente



Características da luz ambiente

- Cria uma luz do tipo ambiente no framework.
- Recebe a cor da luz como parâmetro.
- Contribui igualmente para todas as superfícies visíveis da cena.
- Não depende da direção da luz.
- Ajuda a evitar regiões completamente escuras na cena.

A iluminação ambiente simula a luz indireta espalhada no ambiente.

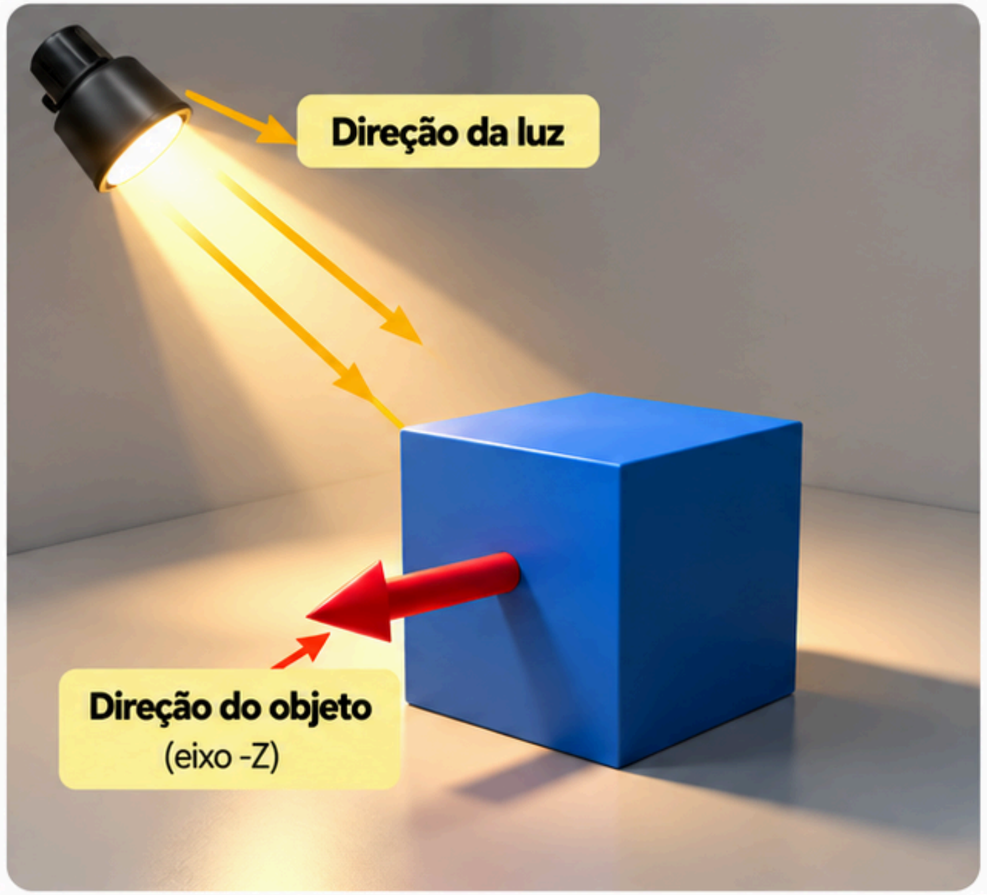
COMPUTAÇÃO GRÁFICA

Direção dos objetos 3D

As funções getDirection e setDirection controlam a orientação.

Métodos principais

```python
class Objeto3D:
    def getRotationMatrix(self):
        # Retorna a matriz de rotação
        return self.transform.getRotationMatrix()

    def getDirection(self):
        # Direção local para frente (eixo -Z)
        forward = [0, 0, -1]
        return self.getRotationMatrix() @ forward

    def setDirection(self, direction):
        # Redefine a orientação do objeto
        position = self.getPosition()
        target = position + direction
        self.lookAt(target)
```



Como funcionam

- **getDirection**
  Retorna a direção atual do objeto.
- **setDirection**
  Redefine a orientação do objeto.
- A direção está associada ao eixo local negativo Z (0, 0, -1).
- Essas funções são úteis para luzes direcionais e objetos 3D.

A orientação do objeto ou da luz influencia diretamente a direção dos raios na cena.

COMPUTAÇÃO GRÁFICA

Luz direcional em Python

Representa uma fonte com direção constante e raios paralelos.

Classe da luz direcional

```
class DirectionalLight(Light):
    def  _init__(self, color=[1,1,1],
                 direction=[0,-1,0]):
        super().__init__(Light.DIRECTIONAL)
        self.color = color
        self.setDirection(direction)
```

`color`	define a cor da luz direcional (em formato RGB).
`direction`	vetor que indica a direção da luz (por exemplo, `[0, -1, 0]`).
`setDirection()`	orienta a luz com o vetor informado.

Características da luz direcional

- Cria uma luz do tipo direcional no framework.
- Recebe a cor da luz e um vetor de direção.
- Usa `setDirection(direction)` para orientar a luz.
- Simula fontes distantes, como o Sol.
- Não possui atenuação com a distância.

A luz direcional é adequada para representar fontes distantes com direção constante.

COMPUTAÇÃO GRÁFICA

Luz pontual em Python

Representa uma fonte localizada com atenuação pela distância.

Classe da luz pontual

```python
class PointLight(Light):

    def __init__(self, color=[1,1,1],
                 position=[0,0,0],
                 attenuation=[1,0,0.1]):
        super().__init__(Light.POINT)
        self.color = color
        self.setPosition(position)
        self.attenuation = attenuation
```

Características da luz pontual

- Cria uma luz do tipo pontual no framework.
- Recebe cor, posição e coeficientes de atenuação.
- Usa `setPosition(position)` para definir a localização.
- A intensidade diminui conforme a distância aumenta.
- Simula lâmpadas, lanternas e fontes próximas.

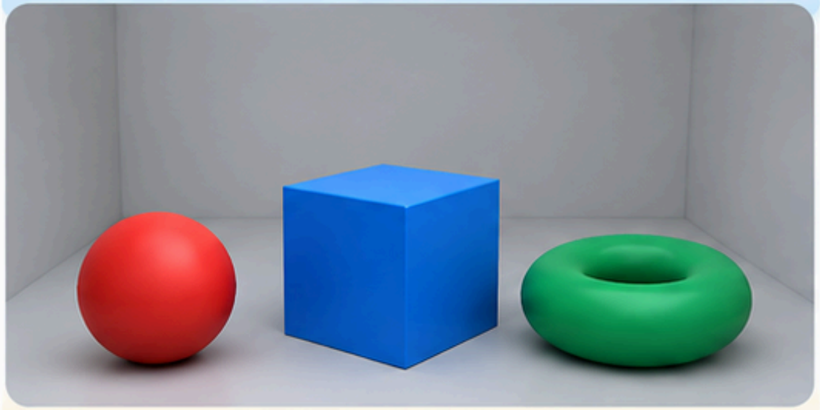
A luz pontual representa fontes próximas cuja intensidade depende da distância.

COMPUTAÇÃO GRÁFICA

Comparando as classes de luz

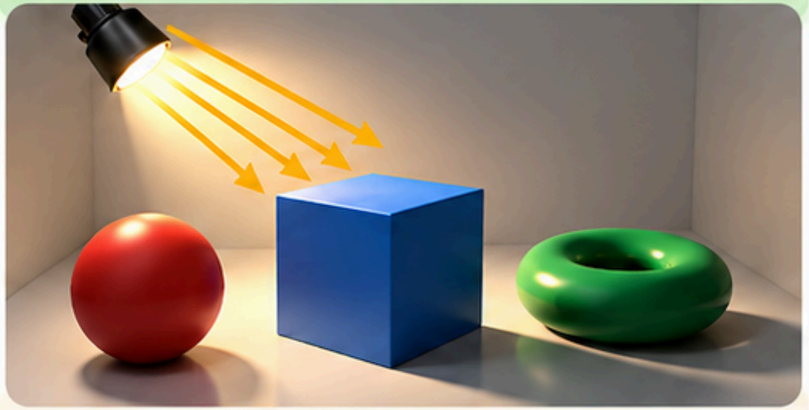
Cada tipo de luz possui propósito e comportamento próprios.

AmbientLight
Iluminação ambiente



- Iluminação uniforme na cena.
- Não depende de posição nem direção.
- Evita áreas totalmente escuras.

DirectionalLight
Luz direcional



- Usa um vetor de direção.
- Raios paralelos.
- Simula fontes distantes.

PointLight
Luz pontual

- Tem posição definida.
- Pode usar atenuação.
- Simula fontes próximas.

Resumo da principal característica	AmbientLight	DirectionalLight	PointLight
	uniforme (ilumina toda a cena)	**direção constante** (raios paralelos)	**fonte localizada** (emite em todas as direções)

A escolha da classe de luz depende do comportamento que se deseja simular na cena.

COMPUTAÇÃO GRÁFICA

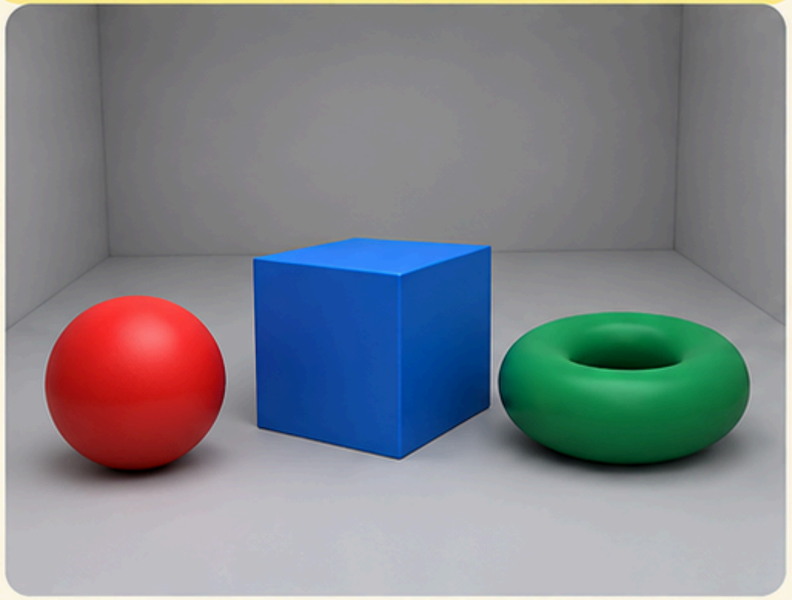
Desafio: qual luz utilizar?

Associe cada situação ao tipo de luz mais adequado.

Escolha entre:

AmbientLight, DirectionalLight e PointLight.

A Quero iluminar toda a cena de forma uniforme, apenas para evitar regiões totalmente escuras.



B Quero simular a luz do Sol incidindo em uma cena externa.

C Quero representar uma lanterna próxima de um objeto.

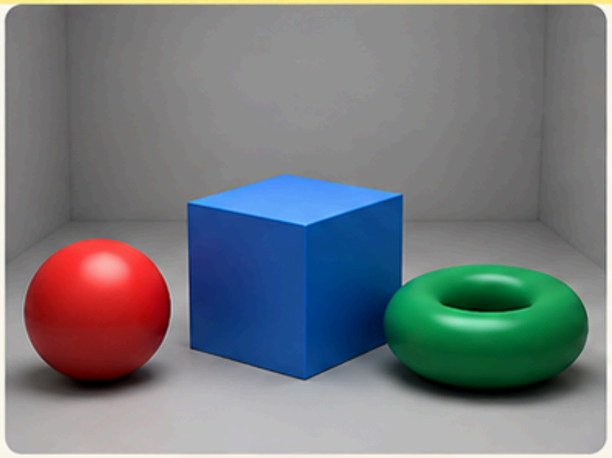
Pense nas propriedades de cada luz: posição, direção e alcance.

COMPUTAÇÃO GRÁFICA

Resolução do desafio

Justificando a escolha do tipo de luz em cada situação.

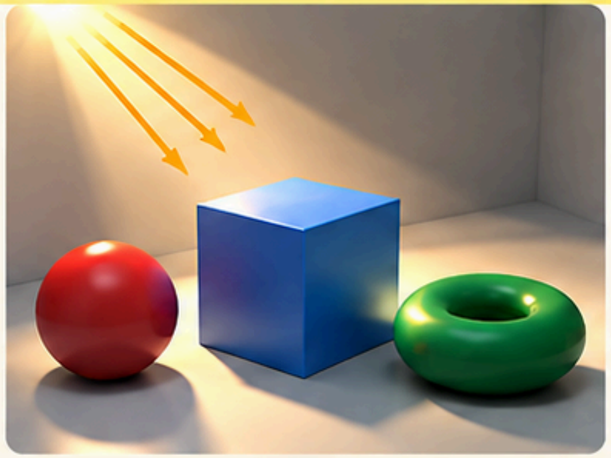
A Resposta: AmbientLight



Justificativa

Fornece uma contribuição uniforme para toda a cena e evita regiões totalmente escuras.

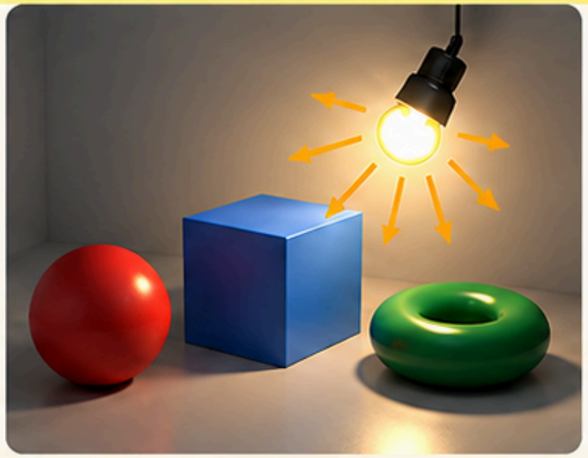
B Resposta: DirectionalLight



Justificativa

Representa fontes muito distantes, com raios paralelos e direção constante, como a luz do Sol.

C Resposta: PointLight



Justificativa

Representa uma fonte localizada, cuja intensidade pode diminuir com a distância, como uma lanterna.

Como decidir

uniforme → AmbientLight

fonte distante → DirectionalLight

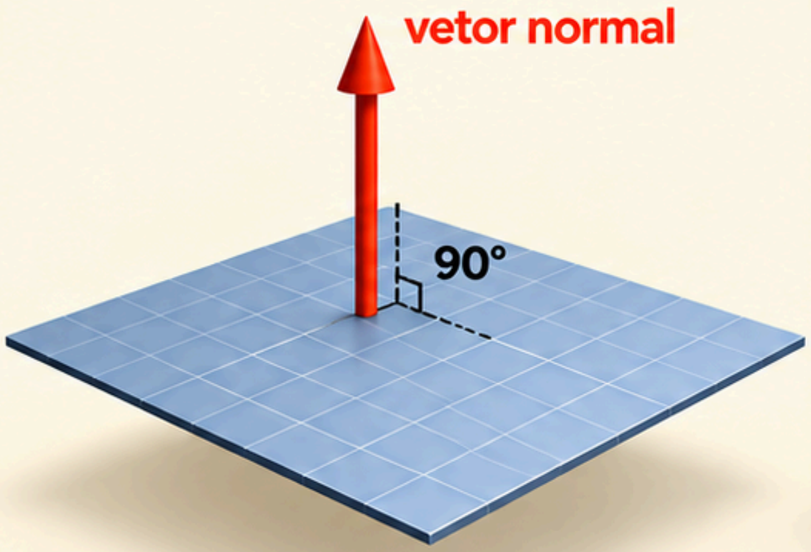
fonte próxima → PointLight

Identificar o comportamento da fonte de luz ajuda a escolher a classe correta no framework.

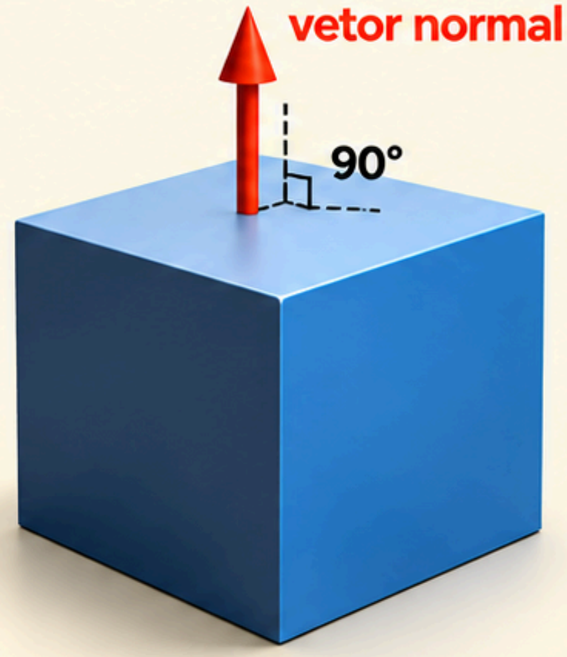
O que são vetores normais?

Um vetor normal é perpendicular à superfície e indica sua orientação.

Plano



Face de um cubo



Principais características

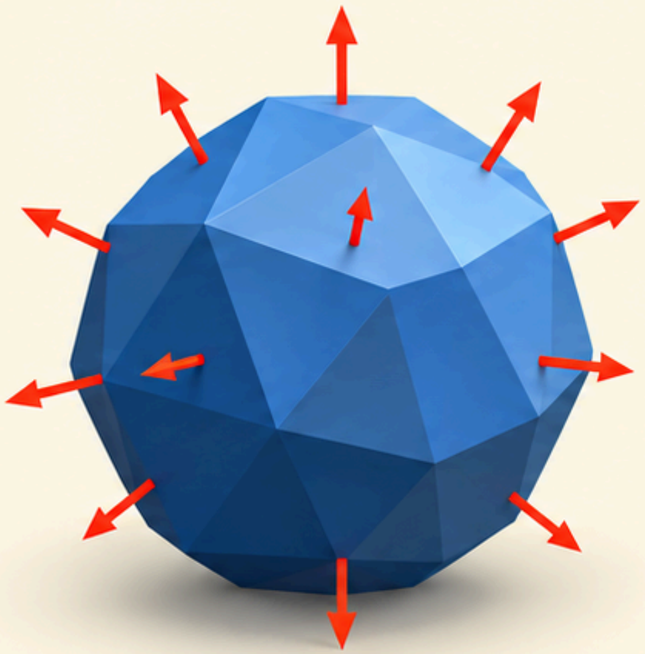
- É perpendicular à superfície.
- Indica a orientação da face.
- É usado nos cálculos de iluminação.
- Pode mudar quando a superfície gira.

Sem o vetor normal, o sistema não sabe como a superfície está orientada.

Normais de faces e de vértices

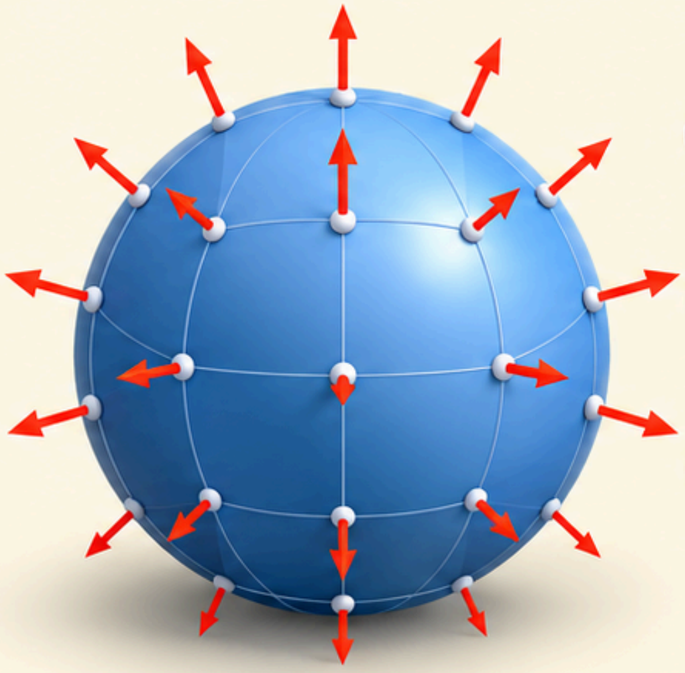
A escolha do tipo de normal influencia diretamente a aparência da superfície.

Normais de face



- Uma normal por face.
- Aparência facetada.
- Comum em superfícies planas.

Normais de vértice



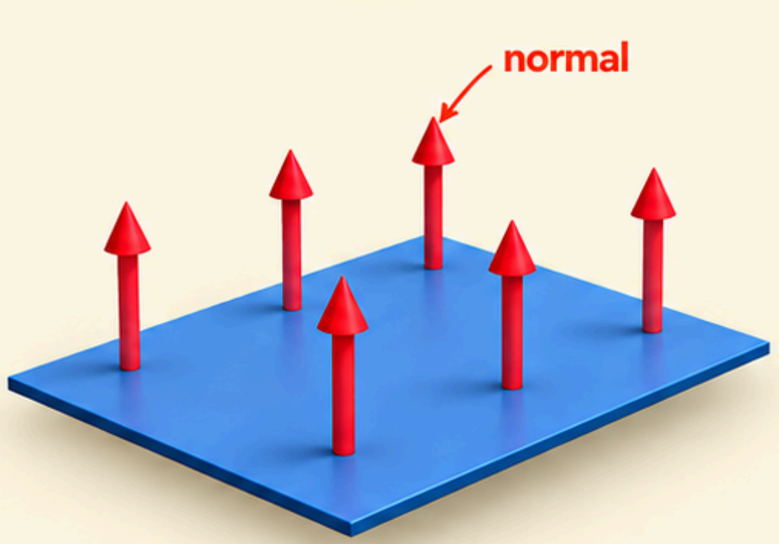
- Uma normal por vértice.
- Transição mais suave.
- Útil em superfícies curvas.

Geometrias semelhantes podem parecer diferentes dependendo das normais utilizadas.

Como visualizar uma normal?

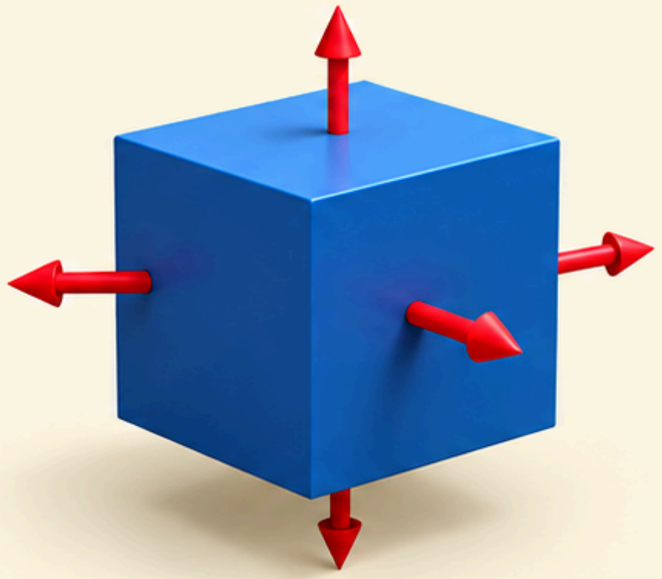
Planos, caixas e superfícies curvas apresentam orientações diferentes.

Plano



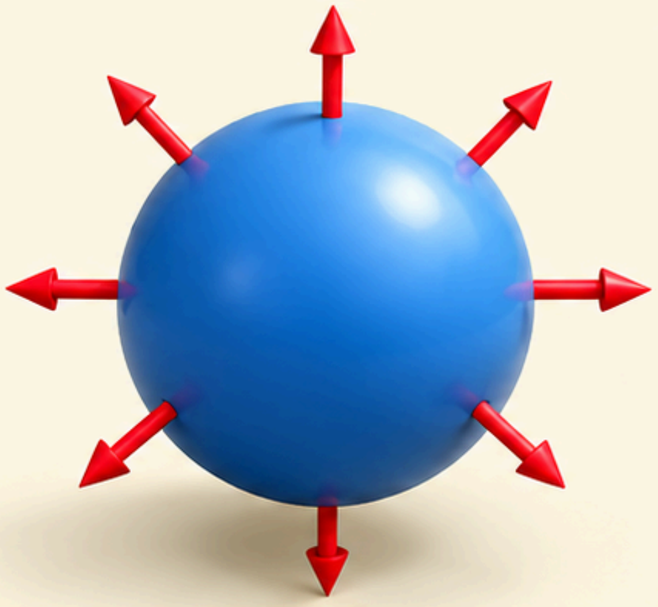
Plano: mesma direção

Caixa



Caixa: uma normal por face

Esfera



Esfera: direção varia ao longo da superfície

A normal sempre representa a orientação local da superfície.

Por que armazenar vetores normais?

O framework precisa dessas informações para calcular corretamente a iluminação.

```
# Dados das normais dos vértices
vertexNormalData = [...]

# Dados das normais das faces
faceNormalData = [...]

# Adiciona os atributos de normais
geometry.addAttribute('vec3',
        'vertexNormal', vertexNormalData)

geometry.addAttribute('vec3',
        'faceNormal', faceNormalData)
```

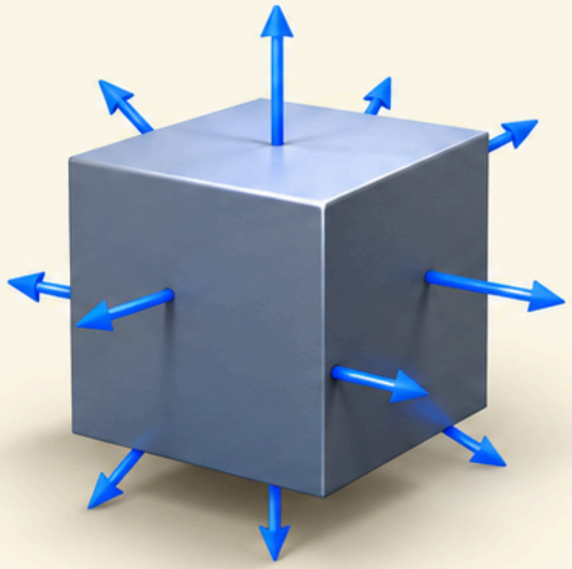
Geometria
Vértices e faces do modelo.

→

Normais
Orientação da superfície.

→

Iluminação
Cálculo da cor em cada ponto.



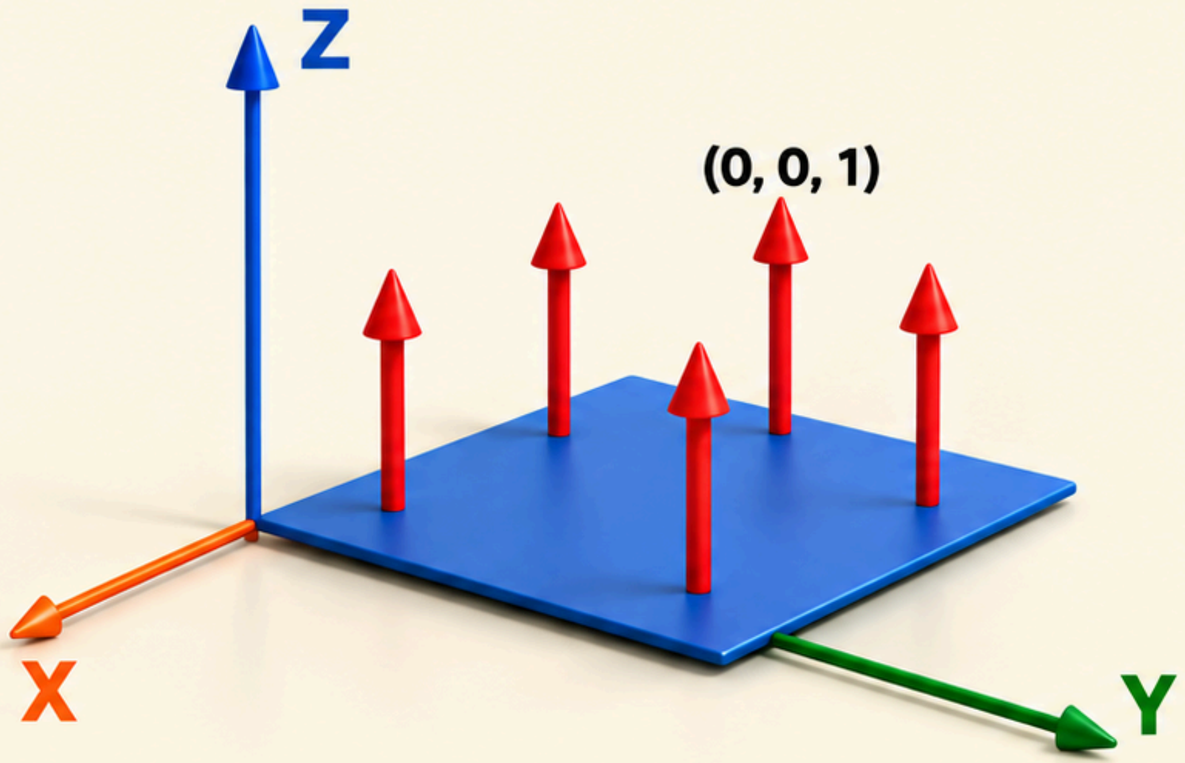
Principais pontos

- Indicam a orientação da superfície.
- Ajudam a calcular luz difusa e especular.
- Podem ser armazenadas por face ou por vértice.
- São enviadas para o shader.

Armazenar normais é essencial para que a iluminação reaja corretamente à geometria.

Vetores normais em retângulos

Em um retângulo no plano XY, as normais apontam para a mesma direção.



Principais características

- Todas as normais têm a mesma direção.
- Face normal = vertex normal.
- A direção é perpendicular ao plano XY.
- Útil para retângulos orientados para +Z.

```
normal = [0, 0, 1]
normalData = [normal, normal, normal, normal]
```

Para um retângulo plano, a orientação é uniforme em toda a superfície.

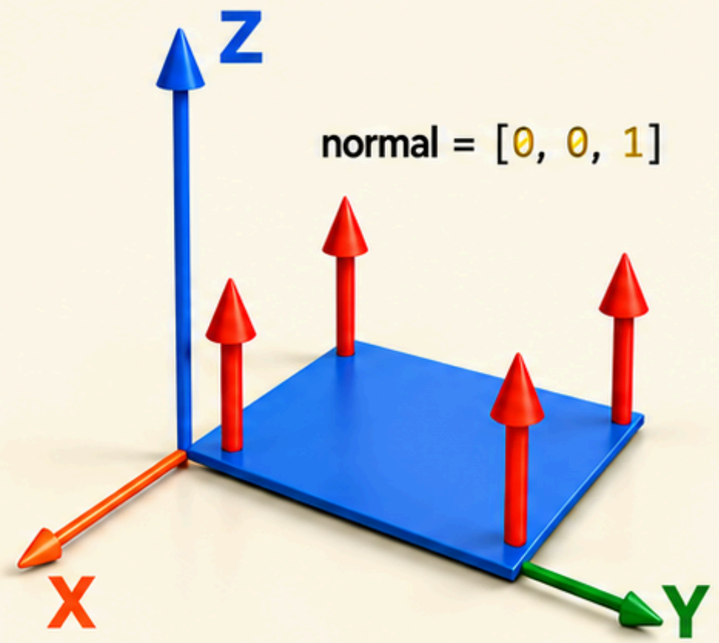
COMPUTAÇÃO GRÁFICA

Normais em Python

O retângulo usa a mesma normal para todos os vértices e para a face.

Código em Python

```
# Normal do retângulo (apontando para +Z)
normal = [0, 0, 1]

# Mesma normal para os 4 vértices
normalData = [normal, normal, normal, normal]

# Adiciona as normais na geometria
geometry.addAttribute('vec3', 'vertexNormal',
normalData)
geometry.addAttribute('vec3', 'faceNormal',
normalData)
```



Principais características

- Todos os vértices do retângulo compartilham a mesma direção de normal.
- As normais dos vértices e da face são idênticas ([0, 0, 1]).
- Os dados de normais são adicionados à geometria.
- A normal aponta para +Z, perpendicular ao plano XY.

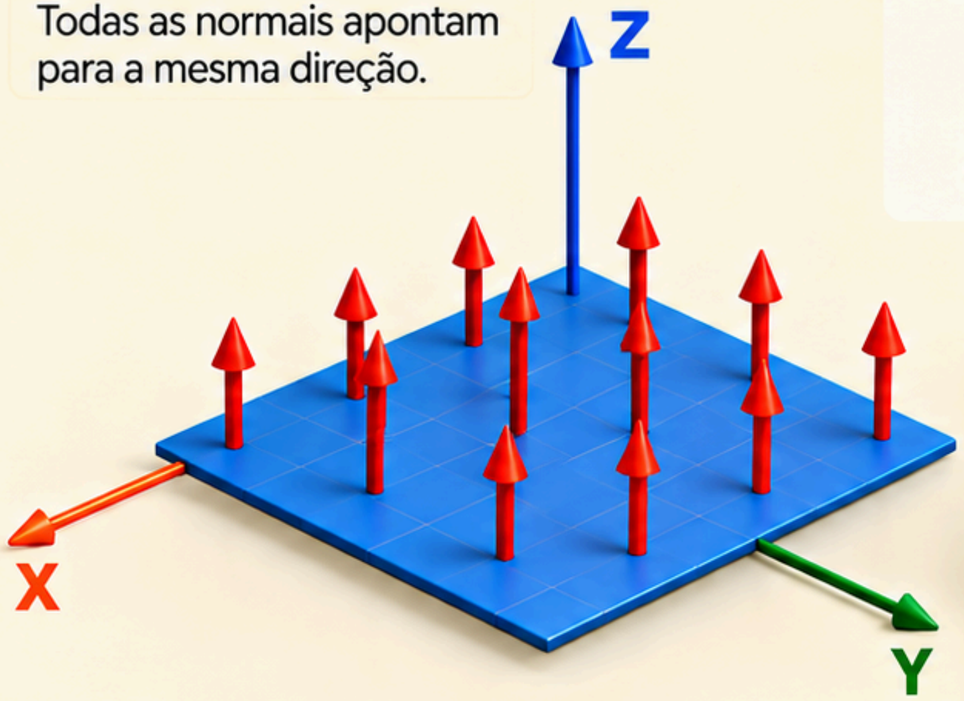
No retângulo plano, a implementação das normais é simples e uniforme.

Interpretando as normais do retângulo

Em uma superfície plana, a orientação é constante em toda a geometria.

Retângulo no plano XY

Todas as normais apontam para a mesma direção.



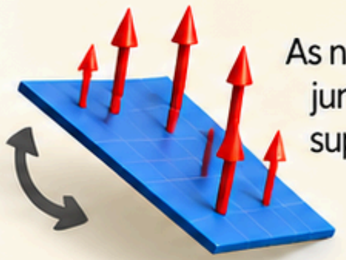
vertexNormal

(0, 0, 1)

faceNormal

(0, 0, 1)

Ao rotacionar o retângulo



As normais giram junto com a superfície.

Principais características

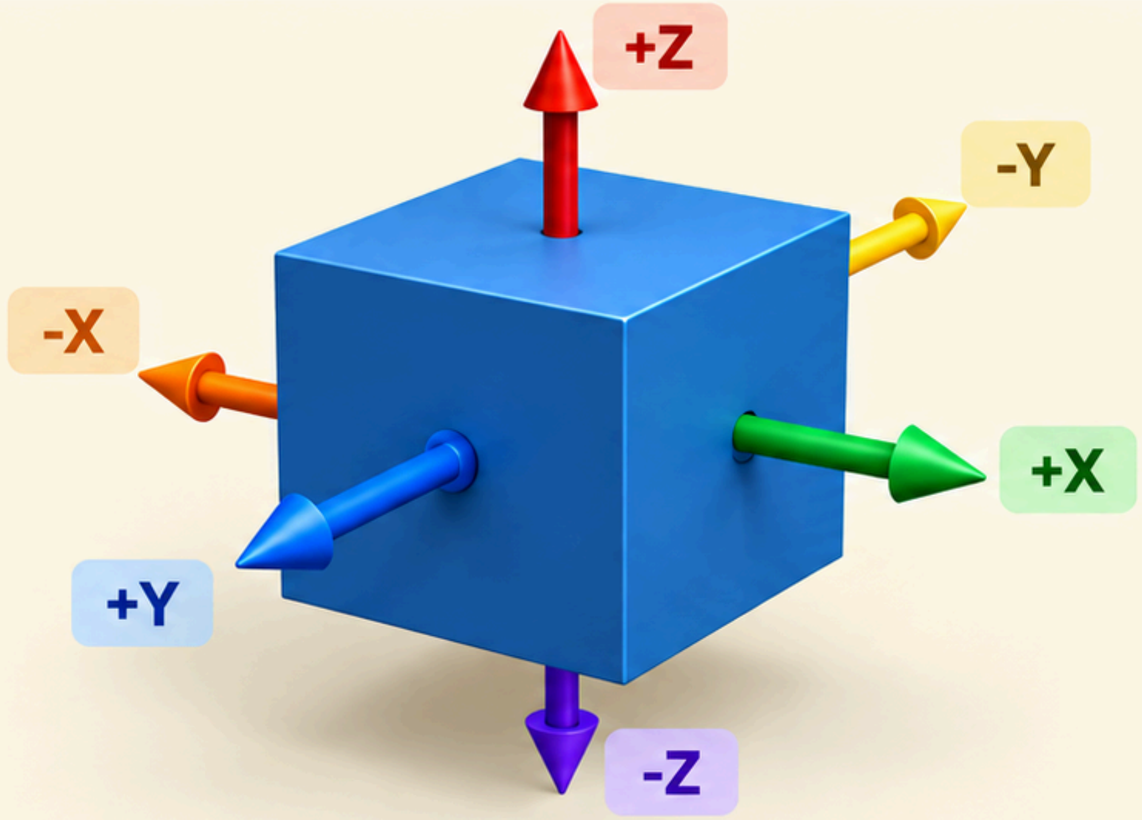
- Todas as normais têm a mesma direção em todos os pontos.
- São perpendiculares ao plano do retângulo.
- São usadas nos cálculos de iluminação difusa e especular.
- Mudam quando o objeto é rotacionado.

Se a superfície é plana e uniforme, sua orientação também será uniforme.

COMPUTAÇÃO GRÁFICA

Vetores normais em caixas

Cada face da caixa possui uma orientação própria.



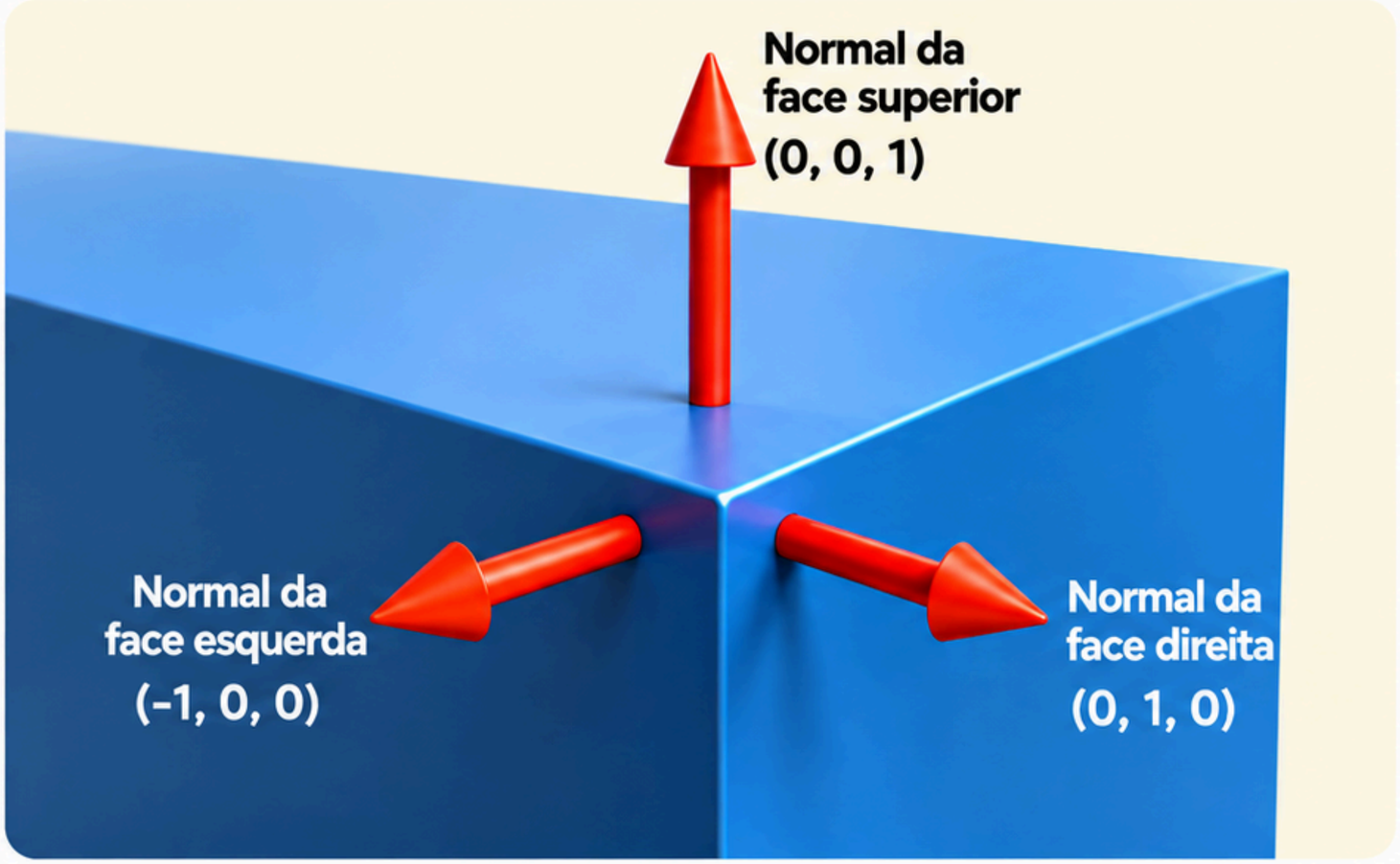
Principais características

- Cada face da caixa possui um vetor normal.
- A aparência permanece facetada (faces planas).
- Faces adjacentes têm orientações diferentes.
- A caixa usa múltiplas posições de vértice repetidas, pois as normais são diferentes em cada face.

Na caixa, a mesma posição geométrica pode participar de faces com normais diferentes.

Um vértice, diferentes normais

No canto de uma caixa, cada face mantém sua própria normal.



Principais características

- Um mesmo canto pertence a várias faces.
- Cada face mantém sua própria orientação.
- Compartilhar a posição não significa compartilhar a normal.
- Isso preserva o aspecto de arestas vivas da caixa.

Mesmo vértice, diferentes normais no dado

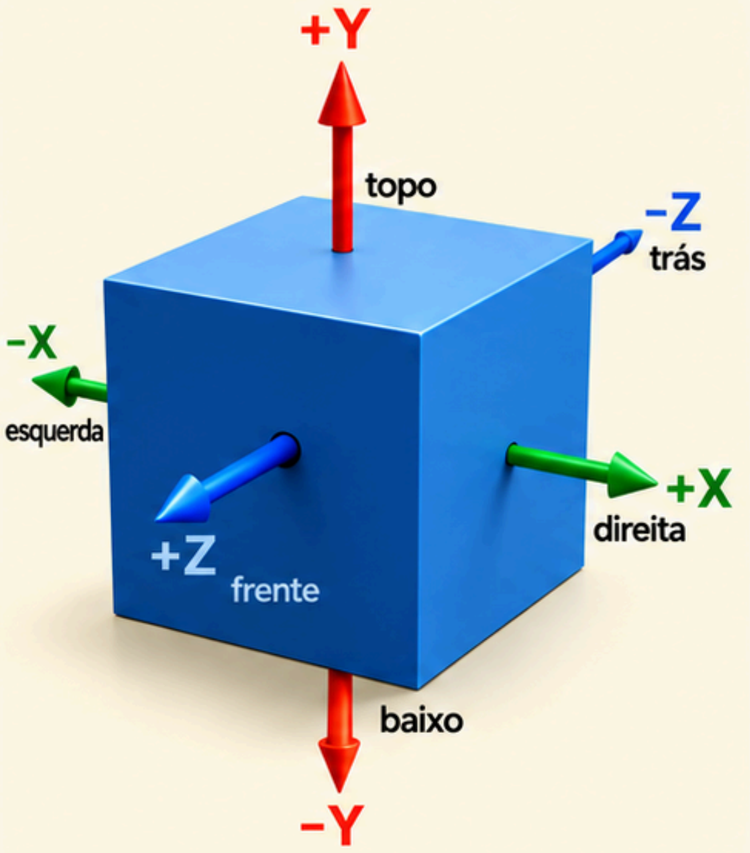
```
Posição (x, y, z)   Normal (x, y, z)
(0, 0, 0)           (1, 0, 0)     # face direita
(0, 0, 0)           (0, 1, 0)     # face superior
(0, 0, 0)           (-1, 0, 0)    # face esquerda
```

Compartilhar posição não significa compartilhar normal.

Implementando as normais de uma caixa

Cada face repete a sua normal para os vértices que a compõem.

```
# Normais das faces de uma caixa (cada face repete a mesma normal)
front  = [0, 0, 1]   # face da frente  (+Z)
back   = [0, 0, -1]  # face de trás    (-Z)
left   = [-1, 0, 0]  # face esquerda   (-X)
right  = [1, 0, 0]   # face direita    (+X)
top    = [0, 1, 0]   # face de cima    (+Y)
bottom = [0, -1, 0]  # face de baixo   (-Y)

# Lista de normais (4 vértices por face, repetindo a normal da face)
normalData = [
    front,  front,  front,  front,    # frente
    back,   back,   back,   back,     # trás
    left,   left,   left,   left,     # esquerda
    right,  right,  right,  right,    # direita
    top,    top,    top,    top,      # cima
    bottom, bottom, bottom, bottom    # baixo
]
```



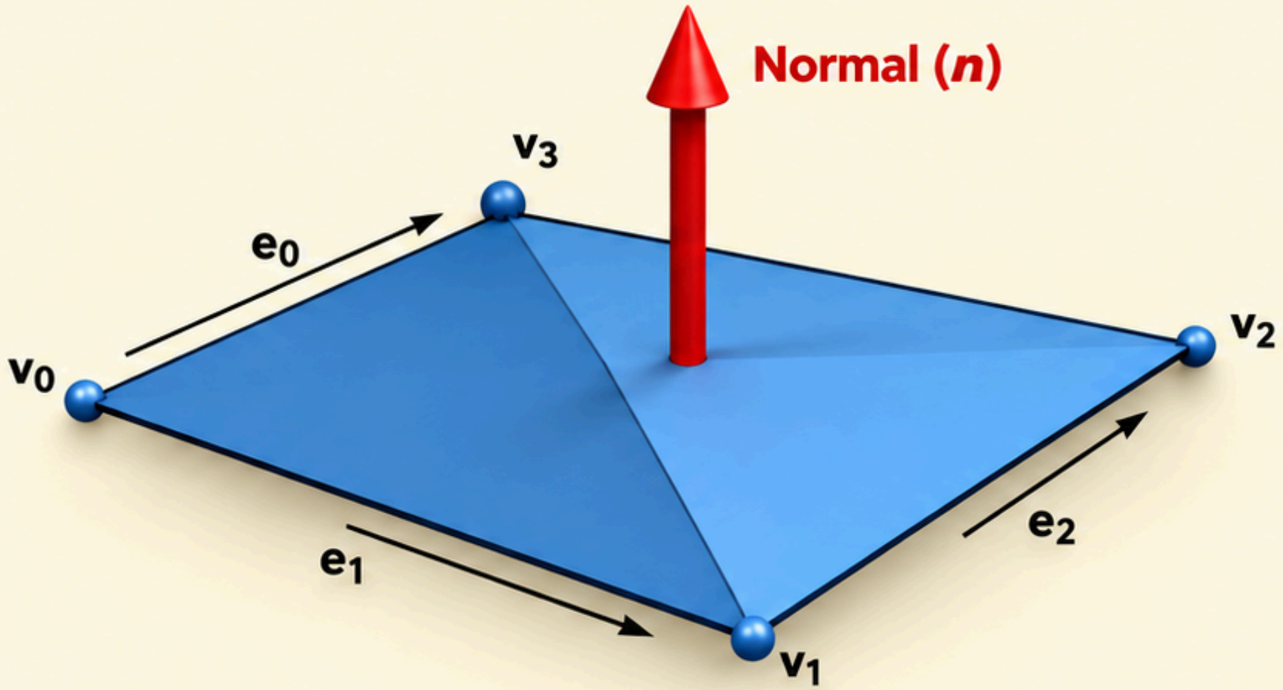
Principais características

- A caixa possui seis orientações principais (+X, -X, +Y, -Y, +Z, -Z).
- Cada face repete a sua normal para os quatro vértices que a compõem.
- Os dados de vértices podem duplicar posições, pois cada face precisa da sua própria normal.
- As normais por face permitem um cálculo de iluminação correto, com arestas bem definidas.

A implementação da caixa organiza as normais por face, preservando as arestas bem definidas.

Vetores normais em polígonos

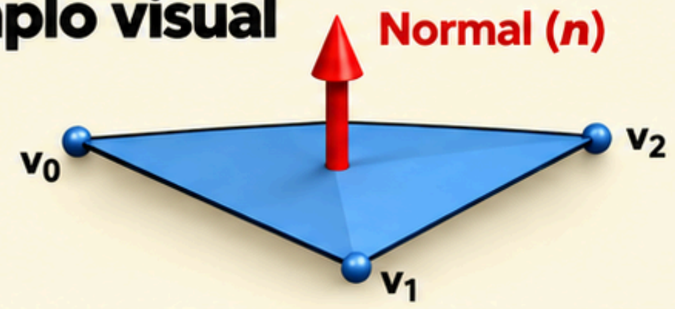
Em polígonos planos, a normal é determinada pela orientação da superfície.



Ideia principal

- Polígonos planos possuem uma direção normal comum.
- A orientação dos vértices influencia o sentido da normal.
- Triângulos e outros polígonos podem usar a mesma lógica.

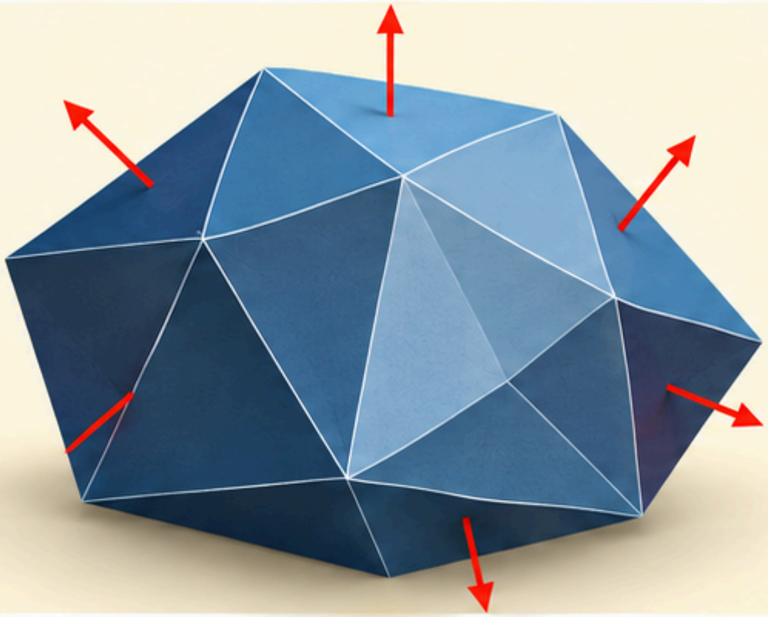
Exemplo visual



A normal de um polígono plano resume a orientação da sua superfície.

Polígonos formados por triângulos

As normais podem ser organizadas a partir das faces que compõem a malha.



```
triangleNormalData = [
    { "v": [0, 1, 2], "normal": [0.0, 1.0, 0.0] },
    { "v": [2, 3, 0], "normal": [0.7, 0.2, 0.6] },
    { "v": [0, 3, 4], "normal": [-0.3, 0.9, 0.1] }
]
```

Principais características

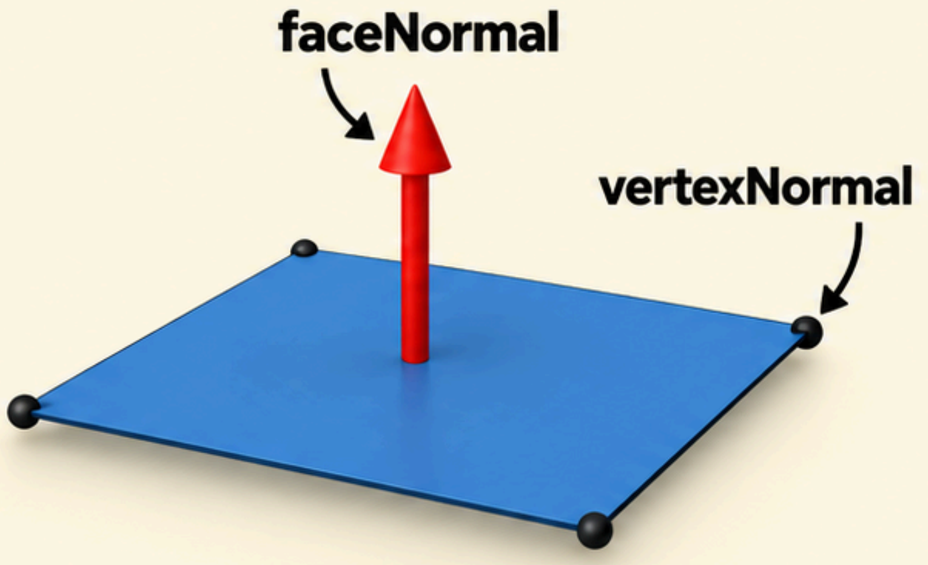
- Uma malha pode ser decomposta em triângulos.
- Cada triângulo pode ter sua própria normal de face.
- Normais consistentes evitam erros de iluminação.
- A triangulação facilita o processamento geométrico.

Triangular a malha ajuda a definir e organizar as normais do polígono.

Implementando normais em polígonos

Os dados de normalData acompanham a orientação usada na geometria.

```
# Normal de um polígono plano
normal = [0, 0, 1]

# Mesma normal para todos os vértices
normalData = [
    normal,
    normal,
    normal,
    normal,
    normal
]
```



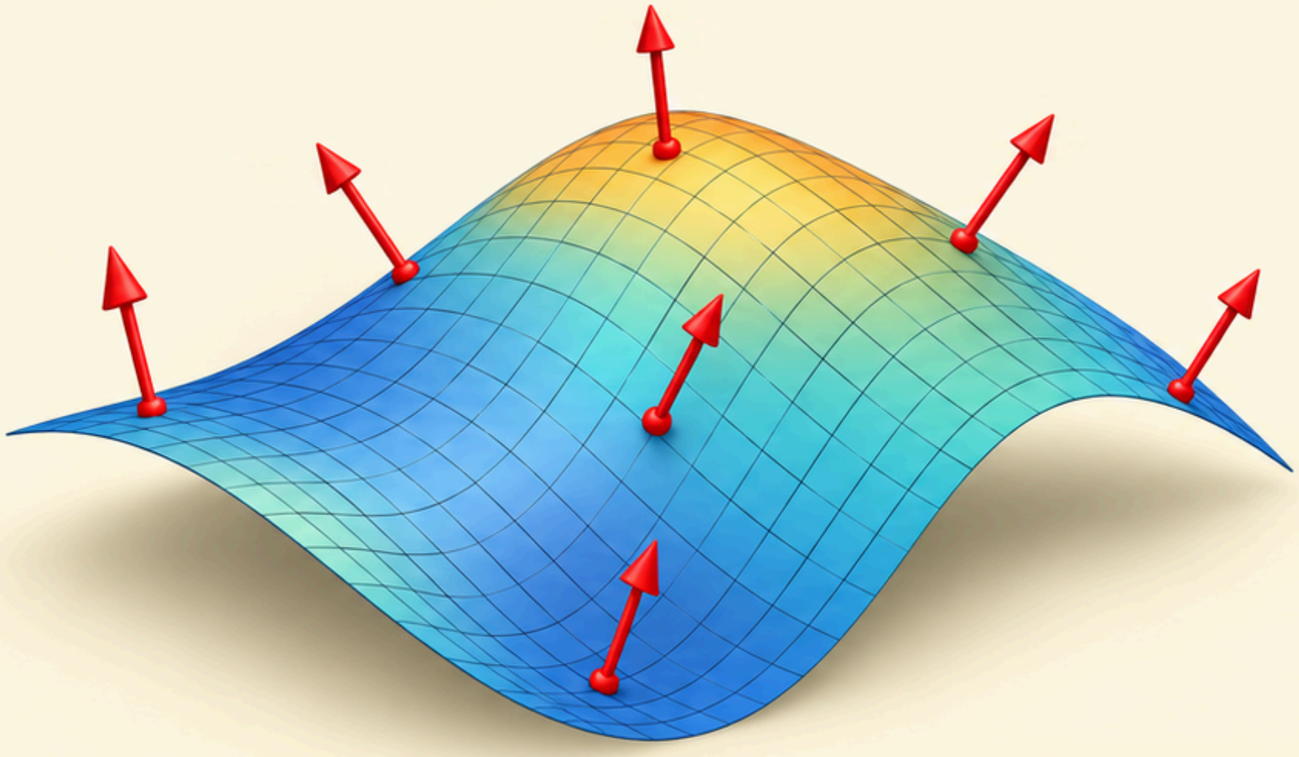
Observações

- A mesma normal pode ser repetida em polígonos planos.
- A orientação deve ser consistente.
- Esses dados serão usados na iluminação.

Em polígonos planos, a implementação das normais costuma ser simples e uniforme.

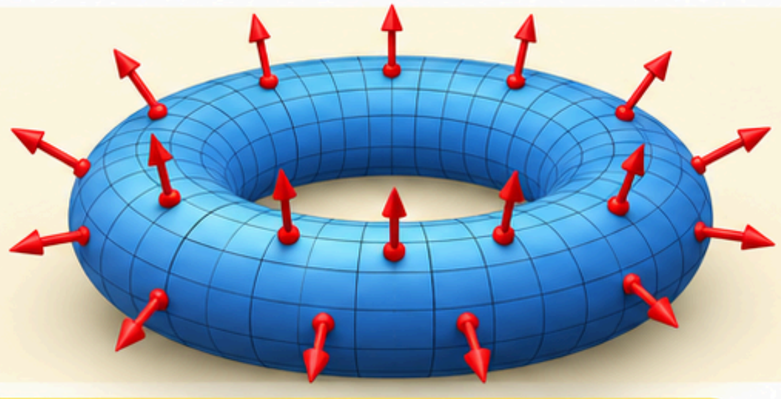
Normais em superfícies paramétricas

Em superfícies curvas, a direção normal varia de ponto para ponto.



Ideia central

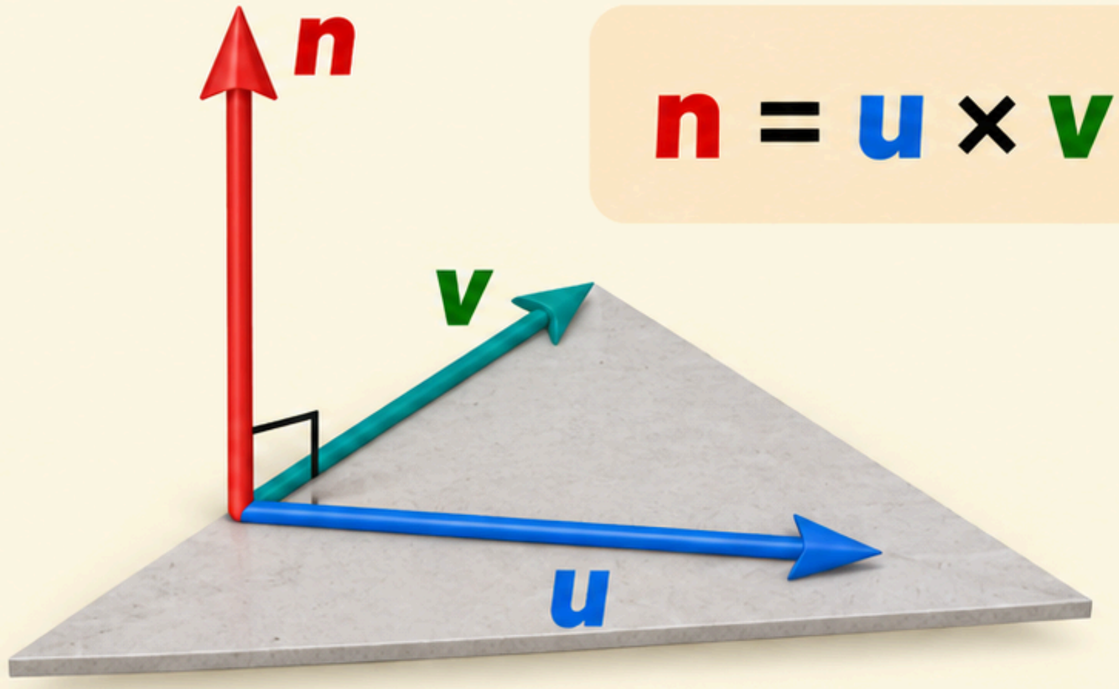
- Superfícies curvas não têm uma única normal global.
- Cada ponto pode ter uma orientação local diferente.
- As normais ajudam a produzir iluminação suave.
- A geometria local define a direção da normal.



Quanto mais curva a superfície, mais as normais mudam ao longo dela.

Produto vetorial: interpretação visual

Duas direções da superfície podem gerar uma normal perpendicular.



$$n = u \times v$$

Principais características

- O produto vetorial usa dois vetores da superfície.
- **O resultado** é perpendicular ao plano definido por eles.
- **O sentido** depende da ordem dos vetores.
- **Esse processo** é útil para calcular normais em superfícies.

O produto vetorial transforma duas direções da superfície em uma direção normal.

CONTATO

rodrigofernandes@unipac.br

OBRIGADO!!